Fuzzy Logic Framework Overview

Developer Reference Guide

Credits

A Fuzzy Logic library is provided for use in this assignment. The original implementation was developed by `t0chas` on GitHub, inspired by Buckland's *Programming Game AI by Example*. Source repository: https://github.com/t0chas/FuzzyLogic.

This framework includes significant modifications to improve API usability, consistency, and readability. It also introduces an alternate fuzzy rule syntax designed to simplify the definition of human-readable rules originally developed by previous OMSCS student, Bob Kerner.

Declaring Fuzzy Variables

Each fuzzy variable should be declared as an `enum` type. The enumeration defines the linguistic (qualitative) labels for that variable.

```
enum FzInputLanePosition { Left, Center, Right };
```

For example, a fuzzy input variable representing distance might include `Near`, `Medium`, and `Far`. A fuzzy output variable for steering might include `TurnLeft`, `Straight`, and `TurnRight`.

It is good practice to distinguish input and output variables clearly:

- `FzInputBallPosZ`
- `FzOutputHeadRotX`

Each enum value corresponds to one membership function within a fuzzy set. The enum defines conceptual categories; the fuzzy set defines how they map to numerical (crisp) values.

Declaring Degree of Membership (DoM) Functions

The framework supports common DoM function shapes such as **Shoulder**, **Trapezoid**, **Triangle**, and **Discrete**. To simplify declarations, you can use helper methods like `GenerateCrossfadeFuzzySet` (for continuous variables) and `GenerateDiscreteFuzzySet` (for discrete outputs).

Shoulder Membership Function

```
public ShoulderMembershipFunction(
    float minX,
    float leftPlateauEndX,
    float rightPlateauBeginX,
    float maxX,
    float leftPlateauHeight = 1f,
```

```
    float rightPlateauHeight = 0f
)
```

Used for edge terms such as *Low* or *High*. It produces a plateau on one side and a linear taper on the other.

Static helpers:

`InstantiateLeft()`
: full membership on the left, tapering to 0 on the right.

`InstantiateRight()`
: the inverse.

Trapezoid Membership Function

```
public TrapezoidMembershipFunction(
    float minX, float plateauBeginX, float plateauEndX, float maxX,
    float leftValleyHeight = 0, float plateauHeight = 1,
    float rightValleyHeight = 0
)
```

Defines a trapezoid-style fuzzy membership function with a flat top between `plateauBeginX` and `plateauEndX` at `plateauHeight`, and linear ramps down to `leftValleyHeight` at `minX` and `rightValleyHeight` at `maxX`. Constraints: `minX` $\leq$ `plateauBeginX` $\leq$ `plateauEndX` $\leq$ `maxX`, and `plateauHeight` $\geq \max($`leftValleyHeight`, `rightValleyHeight`$)$. Special cases include an immediate step-up when `plateauBeginX` $=$ `minX` and an immediate step-down when `plateauEndX` $=$ `maxX`. If `plateauBeginX` $=$ `plateauEndX`, the top collapses to a triangle peak. The representative value is $($`plateauBeginX` $+$ `plateauEndX`$)/2$.

Triangle Membership Function

```
TriangleMembershipFunction(float leftX, float peakX, float rightX,
    float leftHeight = 0f, float peakHeight = 1f, float rightHeight = 0f)
```

Represents a triangular DoM defined by left base `leftX`, peak `peakX`, and right base `rightX` with corresponding heights. Membership increases linearly from the left base to the peak, then decreases linearly to the right base. The peak usually has height 1.0; nonzero `leftHeight` and `rightHeight` allow lifted bases. `fX(x)` evaluates membership at x. `RepresentativeValue` returns `peakX`.

Discrete Membership Function

```
DiscreteMembershipFunction(float representativeValue, float degree = 1f)
```

Defines a constant, discrete DoM centered at a single crisp location. `RepresentativeValue` specifies the X position for the linguistic term, and `Degree` sets its membership weight (default 1.0). `fX(x)` always returns `Degree`, since the discrete term has no shape over the domain. This form is primarily used for output variables with Max–Average defuzzification.

GenerateCrossfadeFuzzySet

```
GenerateCrossfadeFuzzySet<T>(params object[] specs)
GenerateCrossfadeFuzzySet<T>(IList<object> specs, IList<float> heights)
where T : struct, IConvertible
```

Builds a `FuzzySet<T>` by crossfading standard DoM shapes across the enum labels of `T`, in enum order. Each item in `specs` is either a single numeric value (float or convertible) denoting a triangle apex, or a pair `(left, right)` denoting a trapezoid plateau. The outermost labels are emitted as shoulders; if an outer label is given only one control point, the shoulder's plateau collapses to that point (triangle-like limit). All adjacent shapes crossfade smoothly: each function's nearest neighbor control point sets the crossfade boundary on that side. If `heights` is supplied, it must match the number of enum values and provides the peak/plateau height per label; omitted heights default to 1.0. After expanding control points, positions must be non-decreasing. Non-float numerics are converted to `float`; a precision-loss warning may be logged.

GenerateDiscreteFuzzySet

```
GenerateDiscreteFuzzySet<T>(params object[] specs)
GenerateDiscreteFuzzySet<T>(IList<object> specs, IList<float> heights)
where T : struct, IConvertible
```

Builds a `FuzzySet<T>` of discrete terms for the enum labels of `T`, in declaration order. Each `specs` item is a numeric representative value (float or convertible) giving the crisp X location for that label. An optional parallel `heights` list sets the initial DoM weight per label; if omitted, all heights default to 1.0. Discrete outputs defined this way are especially suitable for Max–Average defuzzification, where the representative value contributes the crisp meaning and the degree acts as its weight. Non-float numerics are converted to `float`; a precision-loss warning may be logged.

Why discrete terms pair with Max–Average. In a discrete fuzzy set, each term directly specifies its crisp representative value r_k, so the entire meaning of the term is captured by that single value. During Max–Average defuzzification, the final crisp output is simply the weighted average of these representative values:

$$y^\star = \frac{\sum_k \mu_k r_k}{\sum_k \mu_k}$$

where μ_k is the degree of membership (activation) for each label. Because the representative values already encode all necessary crisp information, discrete DoM functions eliminate the need for any internal shape or curve. Only the weights matter.

Rendering Fuzzy Sets

```
RenderFuzzySetAscii<T>(FuzzySet<T> set, int width = 72, int height = 12)
```

Draws an ASCII visualization of the fuzzy set, showing overlapping membership functions. Useful for console debugging and instructional output.

Fuzzy Expressions

Credit: The original extension-style fuzzy rule composition API was developed by previous OMSCS student Bob Kerner.

The domain-specific language (DSL) provides composable builders for antecedent and consequent expressions:

```
If( Or( And(A, B), C, Not(D)) ).Then(Q)
```

Core components:

`If(x)`
: starts an antecedent.

`And(a, b)`, **`Or(a, b)`**, **`Not(x)`**
: standard logic.

`Very(x)`, **`Fairly(x)`**
: linguistic modifiers.

`Nor`, **`Nand`**
: convenience negations.

Expressions are purely declarative: evaluation occurs later during runtime rule processing.

Writing Rules with the DSL

Rules connect input conditions (antecedents) to output actions (consequents). Using the DSL, each call to `If(...).Then(...)` produces a single `FuzzyRule<T>` where `T` is the enum for the *output* variable.

```
FuzzyRule<FzOutputHeadRotX>[] rules = new FuzzyRule<FzOutputHeadRotX>[] {
    If(FzInputBallPosZ.Negative).Then(FzOutputHeadRotX.RotPosDir),
    If(FzInputBallPosZ.Zero)    .Then(FzOutputHeadRotX.NoRot),
    If(FzInputBallPosZ.Positive).Then(FzOutputHeadRotX.RotNegDir),

    If(FzInputBallVelZ.Negative).Then(FzOutputHeadRotX.RotPosDir),
    If(FzInputBallVelZ.Zero)    .Then(FzOutputHeadRotX.NoRot),
    If(FzInputBallVelZ.Positive).Then(FzOutputHeadRotX.RotNegDir),
};
```

Wrap Rules in a `FuzzyRuleSet`

Group rules *per output variable* into a `FuzzyRuleSet<T>`. This simply binds the output variable's fuzzy set (its terms/DoMs) with the rules you wrote. This will be passed on to the Fuzzy Logic framework for rules evaluation and defuzzification.

```
return new FuzzyRuleSet<FzOutputHeadRotX>(headRotSet, rules);
```

Most usage relies on defaults provided by the framework for evaluation, merge, and defuzzification. In the Racetrack assignment, you typically create one rule set for throttle and one for steering, then pass both to `ApplyFuzzyRules` on every call to `UpdateControls()`, as detailed in **Rule Application**.

Important: Rule Coverage and Domain Completeness. Every linguistic value (enum label) of each fuzzy input variable referenced in a `FuzzyRuleSet` must appear at least once in the antecedent side of the rules. If you use logical conjunctions such as `And(...)` or `Or(...)`, ensure that all meaningful combinations of input terms are represented. Likewise, the degree-of-membership (DoM) domain for each variable must cover all possible input values. If any region of the input space lacks rule or DoM coverage, the system may encounter a case where no rule is activated (no membership value or confidence exceeds 0.0), producing undefined crisp outputs during defuzzification.

Fuzzy Evaluation

On every update (each frame in the 3D Ball example, each `UpdateControls()` call in the Racetrack), fuzzify all crisp sensor values into a shared `FuzzyValueSet`:

```
inputBallPosXSet.Evaluate(currBallPosX, fzInputValueSet);
inputBallPosZSet.Evaluate(currBallPosZ, fzInputValueSet);
inputBallVelXSet.Evaluate(ballVelX, fzInputValueSet);
inputBallVelZSet.Evaluate(ballVelZ, fzInputValueSet);
```

Then evaluate rule sets and defuzzify results for control output.

Rule Application

```
ApplyFuzzyRules<FzOutputThrottle, FzOutputWheel>(
    fzThrottleRuleSet,
    fzWheelRuleSet,
    fzInputValueSet,
    out var throttleRuleOutput,
    out var wheelRuleOutput,
    ref mergedThrottle,
    ref mergedSteering
);
```

This helper:

- Evaluates all fuzzy rules.
- Merges results into per-channel output sets.
- Defuzzifies to crisp control values.

Guards against NaN or infinite outputs, ensuring that at least one rule fires for valid inputs.

Technically, `ApplyFuzzyRules()` is not part of the Fuzzy Logic API, however you are required to use this in the Racetrack assignment. Refer to the implementation of `ApplyFuzzyRules()` in the FLBall3DAgent example or Racetrack if you need to implement this functionality in a different project.

Diagnostics

Value Set Dump

```
DiagnosticPrintFuzzyValueSet<T>(FuzzyValueSet set, StringBuilder sb)
```

Lists all membership degrees for a given variable.

Rule Set Dump

```
DiagnosticPrintRuleSet<T>(FuzzyRuleSet<T> rules,
    FuzzyValue<T>[] outputs, StringBuilder sb)
```

Prints rules, their evaluated output degrees, and confidences for inspection.

Debug Sequence Example

```
DiagnosticPrintFuzzyValueSet<FzInputTrackCurvature>(fzInputValueSet, sb);
DiagnosticPrintFuzzyValueSet<FzInputRoadPosition>(fzInputValueSet, sb);
DiagnosticPrintRuleSet<FzOutputThrottle>(fzThrottleRuleSet,
                                         throttleRuleOutput, sb);
DiagnosticPrintRuleSet<FzOutputWheel>(fzWheelRuleSet, wheelRuleOutput, sb);
sb.Append($"throttle: {Throttle}\n");
sb.Append($"steering: {Steering}\n");
vizText.text = sb.ToString();
```

Summary

The complete fuzzy logic workflow can be divided into two phases: setup and runtime.

Setup phase:

1. Define fuzzy variables and their linguistic states (enums) for all fuzzy inputs and outputs.
2. Define degree-of-membership (DoM) functions for each variable, using `GenerateCrossfadeFuzzySet` or `GenerateDiscreteFuzzySet` as appropriate.
3. Define fuzzy rules linking input variable states (antecedents) to output variable states (consequents).

Runtime phase (executed on every update):

a. Fuzzify current sensor or state inputs into a shared `FuzzyValueSet`.
b. Apply fuzzy rules using `ApplyFuzzyRules` to produce crisp outputs.
c. Use diagnostic printing utilities to inspect fuzzified inputs, rule activations, and defuzzified results for debugging or verification.